GEOG5415M Programming for Spatial Data Science

Week 7: Machine Learning in Action (answers)

In this practical we will go through a worked example of machine learning in action. We are going to replicate some of the work in the paper:

- Asher, M., Oswald, Y., & Malleson, N. (2025). Understanding pedestrian dynamics using machine learning with real-time urban sensors. *Environment and Planning B: Urban Analytics and City Science*, 52(8), 1994-2017. DOI:10.1177/23998083251319058

The aim of the work is to build a model that can predict *pedestrian footfall* for a particular hour, given information about the time, the weather, and the built environment.

If you are interested, the original code, in full, is available from the paper's Github repository:

- github.com/nickmalleson/footfall/tree/main/MelbourneAnalysis

We will go through the following steps:

1. Data preparation (including downloading, cleaning and linking)
2. Data analysis (look for missing data and look at data distributions)
3. Model the data (including model selection)

This is the teachers' version of the notebook. Cells marked `# ANSWER` *(code) or* ***ANSWER*** *(text) contain suggested answers. Where an exercise asks for the same thing several times (e.g. three histograms), only one example is given.*

In [ ]:
```
import os
import time

# Normal data science libraries
import geopandas as gpd
import pandas as pd
import seaborn as sns
#from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

# machine learning packages
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV,
KFold
```

```
from sklearn.metrics import mean_squared_error, r2_score

# set seaborn plotting theme to white
sns.set_theme(style="white")
```

In [ ]:
```
# We need this to use the gdf.explore() function
!pip install mapclassify
```

Data Preparation

We need to download, process, clean and merge the following data sources:

- footfall counts (our target)
- weather conditions
- built environment features
- date information (public holidays, school term times, etc.)

If you are interested, the full code is organised into various notebooks in the original repository's PreparingData directory.

Downloading and reading data

Footfall counts

The first dataset we need is the footfall counts -- i.e. the counts of pedestrians who pass Melbourne's footfall cameras every hour. In the paper, we had to find the data on the Melbourne Open Data Portal, download a few different files (one for old counts, one for new ones) and merge them. If you want to see the code to do this in full, have a look at the Data Preparation Folder in the paper's repository. For this practical, we have made the data available with the notebook to save some time, but there is still a lot of cleaning and preparation to do.

The data are stored in the file called "sensor_counts.csv.gz" in the data/week_7 directory. Note that the file has the '.gz' extension. This is short for 'gzip' and it means that the csv file has been compressed using an algorithm called 'gzip' so that it takes up less space. Fortunately pandas makes it really easy to read and write files using the argument `compression='gzip'`. As with other data, we can read it directly from the module's github repository

Run the code below to read the compressed csv file and assign it to a variable called `sensor_counts`.

In [ ]:
```
url_to_sensor_data = "https://github.com/MSc-Urban-Environmental-Leeds/GEOG5415M-Programming-for-Spatial-Data-Science/raw/refs/heads/main/data/week_7/sensor_counts.csv.gz"
sensor_counts = pd.read_csv(url_to_sensor_data, compression='gzip')
```

Have a look at the sensor counts. Each row holds the number of counts from a sensor for a particular hour (the `hourly_counts`) column, as well as some other information. There are quite a lot of rows!

In [ ]:
```
sensor_counts
```

There is a column that we don't need so let's get rid of it. The easiest way to do this is to use the `sensor_counts.drop()` function. For example, if you wanted to remove a column called 'MyColumn' you could remove it with:

```
sensor_counts.drop(columns = ['MyColumn'])
```

Write the code below remove the column called 'Unnamed: 0'. (Hint: don't forget to reassign the output of `drop()` back to your `sensor_counts` variable)

In [ ]:
```
# ANSWER
sensor_counts = sensor_counts.drop(columns=['Unnamed: 0'])
```

The next chunk will check that the column removal worked correctly. If it runs without an error then you have done it right. (Ask someone if you'd like us to explain how it works)

In [ ]:
```
column_still_in_df = [c for c in ['Unnamed: 0'] if c in
sensor_counts.columns]
if column_still_in_df:
    raise ValueError(f"These columns are still in the dataset:
{column_still_in_df}")
print("Column removed successfully")
```

Now we need to check that the remaining columns that we have store their data correctly. Are they stored using an appropriate data type?

Write the code below to see what types of data the columns store. (hint: you can use the `.info()` function to do this).

In [ ]:
```
# ANSWER
sensor_counts.info()
```

Most are OK. The months and days are text, so these are correctly represented as `object` types. The numerical data are all whole numbers, so correctly represented as `int`s.

But what about the `datetime` column? That is also being stored as a string object, which means that pandas won't be able to do reliable date arithmetic (e.g. comparing dates, adding to dates, etc.).

To fix this we can change the type of the column to a `datetime` object using the `pd.to_datetime()` function. Have a look at the documentation. It tells us that the only 'positional' (required) argument is a Series (the `arg` parameter). Often we also need to provide the `format` argument to tell it how to convert the string to a date, but in this case our string date is formatted in a standard way so the function can work out what to do.

If we had a string column called `my_date` in the dataframe called `df` we could convert it like this:

```
df['my_date'] = pd.to_datetime(df['my_date'])
```

Write the code below to convert the column called `datetime` in your `sensor_counts` dataframe to a proper 'datetime' object

In [ ]:
```
# ANSWER
sensor_counts['datetime'] = pd.to_datetime(sensor_counts['datetime'])
```

This next chunk checks that the conversion worked ok

In [ ]:
```
if not pd.api.types.is_datetime64_any_dtype(sensor_counts['datetime']):
    raise ValueError(f"☒The datetime column is incorrectly being stored as a: {sensor_counts['datetime'].dtype}")
else:
    print("✅Datetime column is a datetime object")
```

While we're doing time-related things, let's create columns to represent the month and day of week as a number. This may be more useful later than the text representations.

This is easy in pandas because, once we have created columns that have the type `datetime`, we can use `.dt` to access date- and time-related aspects. To find out what is available through the `.dt` attribute, have a look at the dt docs and, more usefully, follow the link to the DatetimeIndex docs. Under 'attributes' on that page it shows you all the different parts of a date that you can access. For example, `sensor_counts['datetime'].dt.month` gives you the number of the month.

Edit the code below to fill in the variables. Then you will be able to print out the year, the day of the week, and the quarter for the very first row in our dataframe.

In [ ]:
```
# ANSWER
first_row = sensor_counts.loc[[0], :]  # Use loc to access the first row

year = first_row['datetime'].dt.year
# Do the same as above to create variables called 'day_of_week' and 'quarter'
day_of_week = first_row['datetime'].dt.day_of_week
quarter = first_row['datetime'].dt.quarter

print("The date of the first row is:", first_row['datetime'].values)
print("The year, day of week and quarter are:", year.values, day_of_week.values, quarter.values)
```

Three things to note about the code above for those who are interested

1. Did you see that when we created `first_row`, we used an extra set of square brackets around the 0: `first_row = sensor_counts.loc[[0], :]`? If we didn't do this then pandas would realise that we only wanted one row, and instead of returning a DataFrame object it would return a Series.

Series behave slightly differently and means the example wouldn't have worked. So adding the square brackets around the zero (`[0]`) means that we we are giving `loc` a list, rather than a single number. This 'tricks' `loc` into returning a DataFrame (even though that DataFrame only contains one row).

2. In the `print` statement we use `.values`. The is because `.dt` returns a series, and when we `print` that we also get things like the row number, which makes the print statement messy. `.values` means we *only* get the value, so it looks neater.

3. Question for those who really want to understand what's going on: why are our year, day and quarter values surrounded by square brackets when they are printed out?

ANSWER: `.values` returns a NumPy array rather than a single number. Here each array contains just one value, and Python prints arrays inside square brackets.

Use `dt` to create columns to store the weekday and months as numbers

In [ ]:
```
sensor_counts['weekday_num'] = sensor_counts['datetime'].dt.weekday +1
# Add 1 otherwise the days would run from 0 to 6
sensor_counts['month_num'] = sensor_counts['datetime'].dt.month
```

It will also be useful later to have a datetime column that just has the day, not the full minutes and hours. This is so that we can join it to other datasets that have daily resolution (don't worry about this - ask someone if you're not sure).

In [ ]:
```
sensor_counts['datetime_day'] = sensor_counts['datetime'].dt.floor('D')
```

There is one other date-related thing we need to do, related to **cyclical features**. Recall that in the lecture we discussed that times and days have big jumps (e.g. at midnight, from Sunday → Monday and from December → January). To get round this, we create *sine* and *cosine* transformations of those variables. Then the model is able to use two columns to capture each feature without the big jumps.

Run the code below to create new features for the time, month and weekday. Don't worry about how this works, but if you would like to know please ask someone and we can explain it -- the concept isn't hard but the code includes some features that you haven't seen yet.

In [ ]:
```
def add_sin_and_cos_features(df, column_to_transform, period):
    # 'period' is the number of values before the cycle repeats (e.g. 24
hours in a day)
    df['Sin_' + column_to_transform] = np.sin(2 * np.pi *
df[column_to_transform] / period)
    df['Cos_' + column_to_transform] = np.cos(2 * np.pi *
df[column_to_transform] / period)
    return df

sensor_counts = add_sin_and_cos_features(sensor_counts, 'time', 24)
```

```
# hours 0-23
sensor_counts = add_sin_and_cos_features(sensor_counts, 'month_num', 12)
# months 1-12
sensor_counts = add_sin_and_cos_features(sensor_counts, 'weekday_num',
7)  # days 1-7
```

In [ ]:
```
sensor_counts.info()
```

Reducing the size of the data

We have a count of footfall for every hour, on every day, for over 10 years. That is a **lot** of data (more than 5 million rows). Joining it to the other datasets and fitting models to all of it would take a long time and needs more memory than Colab (or a typical laptop) provides. So here we will just keep the data for the year 2015 (chosen arbitrarily). We do this now, before joining the other datasets, so that all of the later steps run quickly.

Run the code below to keep only the counts from 2015.

In [ ]:
```
sensor_counts = sensor_counts.loc[ sensor_counts['datetime'].dt.year ==
2015, :]
print("Number of rows in 2015:", len(sensor_counts))
```

Sensor locations

We have now finished reading and preparing the footfall counts.

We know what the counts at each sensor are, but we don't know *where* the sensors are located. We need to know this so that we can map the sensor counts, and also so that we can join the sensors to other spatial data about the local built environment. The sensor locations are a separate file that we downloaded from the Melbourne Data Portal. For convenience we have put the file on module repository so you can load it easily:

In [ ]:
```
sensor_locations = pd.read_csv("https://github.com/MSc-Urban-
Environmental-Leeds/GEOG5415M-Programming-for-Spatial-Data-
Science/raw/refs/heads/main/data/week_7/sensor_locations.csv")
sensor_locations
```

Again, you need to get rid of that strange column.

Write the code below remove the column called 'Unnamed: 0'..

In [ ]:
```
# ANSWER
sensor_locations = sensor_locations.drop(columns=['Unnamed: 0'])
```

Stretch activity: Convert the `sensor_locations` table to a GeoDataFrame and map it.

In [ ]:
```
# ANSWER (optional stretch activity)
sensor_gdf = gpd.GeoDataFrame(
    sensor_locations,
```

```
    geometry=gpd.points_from_xy(sensor_locations['Longitude'],
sensor_locations['Latitude']),
    crs="EPSG:4326"   # WGS84 (latitude and longitude)
)
sensor_gdf.explore()
```

Merge the sensor counts with the sensor locations

Now that we have counts for the sensors as well as their locations, we can merge the two DataFrames. This is made easy because both datasets have a column called `sensor_id`, which uniquely identifies a sensor.

We will use the `pd.merge` function to merge them. Have a quick look at the pandas.merge() documentation. The function has two *required* parameters: 'left' and 'right'. These are the two datasets that you want to merge. In our case it also needs the 'on' parameter so that it knows which column to use to merge the datasets. For example, if we wanted to merge two datasets, `df_a` and `df_b`, using column `id` as the linking column we would use `merge` like this:

```
merged_df = pd.merge(df_a, df_b, on='id')
```

Edit the code below to create a new variable called `location_counts` by merging the `sensor_locations` and `sensor_counts` dataframes using the `sensor_id` column.

In [ ]:
```
# ANSWER
location_counts = pd.merge(sensor_locations, sensor_counts,
on='sensor_id')
```

In [ ]:
```
# Have a look at your new table to check the columns look OK
location_counts
```

To make sure this worked, let's check that your new `location_counts` dataframe has the same number of rows as the original `sensor_counts` dataframe

In [ ]:
```
if len(location_counts) == len(sensor_counts):
    print("Merged dataframe has the correct number of rows")
else:
    raise ValueError(f"Merged dataframe length {len(location_counts)}
doesn't match the original sensor counts length {len(sensor_counts)}")
```

Weather and holiday data

We need information about when school and public holidays take place, and also information about the weather. These were collected from a few places, see the paper if you're interested.

One thing to note about the code below is that we use the `parse_dates=['date']` argument. This tries to automatically create a datetime object when reading the 'date' column, rather than storing it as a string. It means that we don't

need to convert the column later (like we had to do for the sensor data). The `dayfirst` argument tells pandas that for one of our files the day comes first, not the year. (We could have used this feature when reading the sensor data earlier, but it was good to see how you have to do it manually).

Load weather and holiday data

In [ ]:
```
url_to_data = "https://github.com/MSc-Urban-Environmental-Leeds/GEOG5415M-Programming-for-Spatial-Data-Science/raw/refs/heads/main/data/week_7/"

daily_rainfall = pd.read_csv(url_to_data+"DailyRainfallData.csv", parse_dates=['date'])
public_holidays = pd.read_csv(url_to_data+"publicholidays.csv", parse_dates=['date'])
school_holidays = pd.read_csv(url_to_data+"schoolholidays.csv", parse_dates=['date'], dayfirst=True)
```

Edit the code below to have a look at those three dataframes that you just loaded. Do they *look* OK? Are the data stored in the right format? Are there any duplicate rows? Etc..

ANSWER: The dates have all been read as `datetime64` and the other columns are numbers, so the types are fine. `public_holidays` has one duplicate row: 25 April 2011 appears twice because Anzac Day and Easter Monday fell on the same day. It does not matter here because we only use 2015, but with the full data the merge would duplicate every count on that day, so it would be worth removing with `public_holidays.drop_duplicates()`.

In [ ]:
```
# ANSWER (one way of checking: look at the column types and count duplicate rows)
for df in [daily_rainfall, public_holidays, school_holidays]:
    df.info()
    print("Number of duplicate rows:", df.duplicated().sum(), "\n")
```

In [ ]:
```

```

In [ ]:
```

```

Merge the weather and holiday data with the sensor data

We want to attach these new dataframes to our sensor data. All of our dataframes have a column called `date`, so we can merge by comparing that column to our `datetime_day`.

We can do this using the `left_on` argument, which tells `merge()` that column in the left dataframe that we want to join, and the `right_on` argument for the right dataframe. So in our case we want `left_on="datetime_day", right_on="date"`. (We didn't have to do this earlier because both dataframes had a column called `sensor_id` that we used to merge).

IMPORTANT!. Another thing to note is that there may be some specific dates missing from our weather and holiday data. The default merge behaviour is to drop rows where there is no match. We can tell pandas not to do this by making it a 'left' join. That means keep all the rows in the 'left' dataset, and if it cannot find a matching value then insert 'na' into that row. We can use the `how="left"` argument to `merge()` and make sure that `location_counts` is the first dataset we pass (that one is treated as the 'left' dataset, the other is treated as the 'right' one).

Run the chunk below to see how to attach the daily_rainfall data to the sensor data.

In [ ]:
```
# Do the merge
location_counts = pd.merge(location_counts, daily_rainfall,
left_on="datetime_day", right_on="date", how="left")
# Drop the 'date' column that was part of 'daily_rainfall'. If we don't
do this later merges will become confused
location_counts = location_counts.drop(columns='date')
```

Now edit the chunks below to merge the other two datasets to the sensors data. (Don't forget to drop the 'date' column after each merge).

In [ ]:
```
# ANSWER
location_counts = pd.merge(location_counts, public_holidays,
left_on="datetime_day", right_on="date", how="left")
location_counts = location_counts.drop(columns='date')
```

In [ ]:
```
# ANSWER
location_counts = pd.merge(location_counts, school_holidays,
left_on="datetime_day", right_on="date", how="left")
location_counts = location_counts.drop(columns='date')
```

Now use `.info()` to check that the columns in the new `location_counts` dataframe look OK. Do you have the right columns and are they of the expected type?

In [ ]:
```
# Check the data have the columns we're expecting:
location_counts.info()
```

One other thing we need to do. Our public and school holiday input data only contained dates that are holidays. Not those dates that are not holidays. So after the merge, all the days that are holidays will have a value of 1, and those that aren't holidays will have 'na'. We can convert those NAs to 0 to show that it isn't missing data, it's a day that isn't a holiday:

In [ ]:
```
location_counts['public_holiday'] =
location_counts['public_holiday'].fillna(0)
location_counts['school_holiday'] =
location_counts['school_holiday'].fillna(0)
```

P.S. Before moving on, here's a hint with merging data: Once we ran the code `location_counts = pd.merge(...)` we created a new `locations_counts`

dataframe and deleted the old one. This means that if we had done something wrong with the merge, we would have to go back and re-read the sensor data, re-merge it to the locations, etc. It's not a problem, but it can be annoying, especially if some of the earlier merge operations took a long time. So, if you want to check whether the merge looks OK before doing it, you can just call the `pd.merge()` function but not assign the resulting new dataframe to the `location_counts` variable. For example you could do this:

```
temporary_df = pd.merge(location_counts, daily_rainfall,
left_on="datetime_day", right_on="date", how="left")
```

and then analyse `temporary_df` to see if the merge worked first. Or, if you just do:

```
pd.merge(location_counts, daily_rainfall, left_on="datetime_day",
right_on="date", how="left")
```

pandas will helpfully show you what the result of the merge will look like (although it doesn't save the result, so you can't do any further analysis)

Check the merges worked OK - do we have missing data?

Just because the columns exist in the merged dataframe doesn't mean that the merge actually worked correctly. There may be some rows where pandas couldn't find a match.

A nice easy way to check whether the merge was OK is to see whether any 'na' values were introduced after the merge. This happens when there are some rows in one dataset that pandas can't find a partner for in the other data. We've seen the combination of `isna()` and `sum()` functions before.

Run the code below. Are there any rows with missing data?

ANSWER: Yes. About 15,000 rows have no rainfall value (`rainfall_mm`), because the rainfall data have some days missing. (The holiday columns also have lots of NAs, but we have just explained that these are days that are not holidays.)

In [ ]:
```
location_counts.isna().sum()
```

The missing rainfall data are a problem. In a real setting we would need to explore this in detail. We could start with making a table that just contained the missing rows and looking for patterns, e.g.:

```
location_counts.loc[location_counts['rainfall_mm'].isna(), :]
```

and then we could try to impute the rainfall. However for now we are just going to make the missing values 0.0 (not good practice!!)

In [ ]:
```
location_counts['rainfall_mm'] =
location_counts['rainfall_mm'].fillna(0)
```

In [ ]:
```
# Now there are no missing values for the columns that matter
location_counts.isna().sum()
```

Load and merge the built-environment spatial data

Now we want to join in other data that may influence pedestrian footfall. Things like hospitals, universities, train stations, etc., etc.

Preparing the spatial data is quite laborious. We downloaded OpenStreetMap data and then used the spatial analysis functions available in geopandas to find the number of different types of objects (like trees, stations, bike stands, bus stops, etc.) within a certain radius of each sensor. If you are interested in how we did this have a look at the FindFeaturesInProximityToSensors.ipynb script in the paper's repository.

For this practical, we will just load the pre-processed data and join it to the sensors. The data that we will read is a table that has, for each sensor, the numbers of different types of physical features within a certain distance (400m in our case)

In [ ]:
```
num_features_near_sensors = pd.read_csv("https://github.com/MSc-Urban-Environmental-Leeds/GEOG5415M-Programming-for-Spatial-Data-Science/raw/refs/heads/main/data/week_7/num_features_near_sensors_400.csv")
num_features_near_sensors
```

Write the code below to merge the `location_counts` data frame with the `num_features_near_sensors` dataframe, using the `sensor_id` column that is common to both dataframes..

In [ ]:
```
# ANSWER
location_counts = pd.merge(location_counts, num_features_near_sensors, on='sensor_id')
```

In [ ]:
```
# This checks you did the merge correctly by checking that the 'trees' column exists and isn't NA
# 'trees' is just one of the columns from the num_features_near_sensors data, we could have checked any of them
if not "trees" in location_counts.columns:
    raise ValueError("There is no 'trees' column after merging, something has gone wrong.")
else:
    print("There is a 'trees' column after the merge")

# If we get here then no error was raised so presumably the trees column exists. Just check it is not NA
if location_counts['trees'].isna().all():
    raise ValueError("'trees' column exists but all values are NA — the merge may have failed.")
else:
```

```
    print("Merge successful: 'trees' column found and contains valid
data.")
```

Data Analysis

Before modelling, we need to analyse our data. Using a combination of statistical analysis and visualisations (using the code you wrote in Week 4), write some code below to get a better understanding of your data. You can be inventive and explore the different variables in any way that you want to, but if you're not sure what to do then you can follow the prompts.

Plot a histogram of the hourly counts.

In [ ]:
```
# Histogram for hourly counts
sns.histplot(location_counts['hourly_counts'])
```

Plot histograms for three additional variables. Are these all OK?

ANSWER: The variables are not normally distributed. The counts and rainfall are very skewed (most hours have low footfall, and most days have little or no rain), and the built-environment variables only take one value per sensor. This is fine for a random forest, which does not assume any particular distribution, but it is one reason why linear regression struggles.

In [ ]:
```
# ANSWER (one example is enough; any numerical column will do)
sns.histplot(location_counts['rainfall_mm'])
```

In [ ]:
```
# ANSWER: see above
```

In [ ]:
```
# ANSWER: see above
```

Do a joint scatter plot for the hourly counts and the rainfall (Hint: you can use `sns.jointplot()` with `kind='scatter'`).

In [ ]:
```
# ANSWER
sns.jointplot(x='rainfall_mm', y='hourly_counts', kind='scatter',
data=location_counts)
```

Well that isn't very clear. Try a kde plot instead. KDE plots are slow to draw with lots of data, so use a random sample of 10,000 rows: `data=location_counts.sample(10000)`.

In [ ]:
```
# ANSWER
sns.jointplot(x='rainfall_mm', y='hourly_counts', kind='kde', fill=True,
data=location_counts.sample(10000))
```

Do box plots to compare the variables: bikes; landmarks; memorials; transport_stops. Hint: an easy way to do this is select the columns that we want using `loc` and pass that to the `sns.boxplot()` function:

```
sns.boxplot(data=location_counts.loc[:,
['var1','var2','var3','var4']])
```

In [ ]:
```
# Code for four boxplots

sns.boxplot(data=location_counts.loc[:, ['bikes', 'landmarks',
'memorials', 'transport_stops']])
```

Finally, this plot adds up the total footfall over the course of a day and plots the daily footfall for each sensor for the month of May. It's messy but you can start to see some of the trends emerge

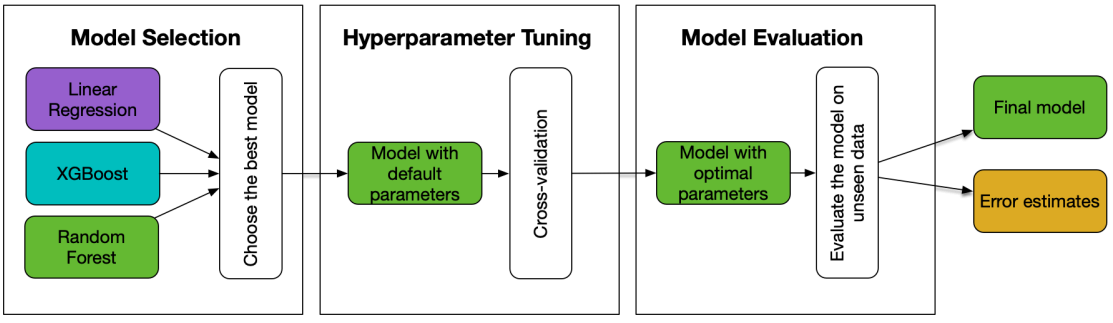
In [ ]:
```
# Sum hourly counts per day per sensor
daily =
location_counts.loc[location_counts.month_num==5,:].groupby(['datetime_d
ay','sensor_id'])['hourly_counts'].sum().reset_index()

# Line plot: one line per sensor
sns.lineplot(
    data=daily,
    x='datetime_day',
    y='hourly_counts',
    hue='sensor_id',
    palette='tab10',      # qualitative colour scheme
    linewidth=1,          # thinner lines
    legend=False          # 43 sensors would make the legend bigger than
the plot
)
plt.xticks(rotation=45)  # rotate the dates so they don't overlap
```

Modelling

Now we are ready to do some modelling. Recall from the lecture that we need to do three things:

1. **Model Selection** to choose the best *type* of model
2. **Hyperparameter tuning** to find the best parameters for our chosen type of model
3. **Model evaluation** to see how well our optimised model works on data that it hasn't seen before.



Data Preparation

Before continuing, let's get our *X* (predictors) and *y* (target) datasets ready. This mostly consists of just choosing the rows and columns in the `location_counts` data frame that we want to use for modelling.

Let's start by selecting the rows that we want. Remember that each row holds the footfall count for a particular hour in our study period. During and after COVID, footfall patterns changed dramatically, so it is unlikely that a model will be able to predict footfall under *normal* conditions as well as during the COVID period. Therefore we will remove the COVID period from our training data. (We are only using data from 2015, so this makes no difference here, but it would if you used the full dataset.)

Also remember that there is a convention to call our data *X* and *y*, so we will do that

Start with the *y* data (our target). Use `loc` to keep only the rows before 2020 (the rows) and select only the 'hourly_counts' column.

As a reminder, the syntax of `loc` is like this:

```
df.loc[ <rows_to_filter>, <columns_to_select> ]
```

In [ ]:
```
y = location_counts.loc[ location_counts['datetime'] < '2020',
'hourly_counts']
```

Now do the same for the *X* dataset (our predictors). This is slightly more complicated though because rather than just selecting one column (the 'hourly_counts') we want to select all the columns that we think might be related to footfall. Let's define the columns we want in their own list to make things easier. There is no 'correct' list here. If you're interested you could come back and add or remove variables from this list and see how it affects the model (remember that `location_counts.columns` will show you all the columns that are available).

In [ ]:
```
# Choose the columns we want to use as our predictor variables, making a
big list
predictor_columns = [
    # The time-related variables:
    'Sin_time', 'Cos_time',
    'Sin_month_num', 'Cos_month_num',
    'Sin_weekday_num', 'Cos_weekday_num',
    # Weather-related:
    'rainfall_mm',
    # Holidays:
    'public_holiday', 'school_holiday',
    # Built-environment
    'lights', 'street_inf', 'bikes', 'landmarks', 'memorials', 'trees',
    'transport_stops', 'bus-stops', 'tram-stops', 'metro-stations',
'taxi-ranks',
    'big-car-parks', 'buildings_2019','avg_n_floors_2019'
]
```

Now we can use `loc` like we did before.

Complete the code below to use the `loc` accessor to choose all rows before 2020 and the columns we defined in the `predictor_columns` list.

In [ ]:
```
# ANSWER
# Create the 'X' variable with rows before 2020 (as above) and columns
from the 'predictor_columns' list

X = location_counts.loc[ location_counts['datetime'] < '2020',
predictor_columns]
```

Have a look at the two new dataframes (*X* and *y*). Do they have the right columns? Do they have the same number of rows?

Do a couple of checks below to see that *X* and *y* are as you expect.

In [ ]:
```
# ANSWER (some possible checks)
X.head()
```

In [ ]:
```
# ANSWER
print(X.shape, y.shape)
```

In [ ]:
```
# ANSWER
X.isna().sum().sum()  # should be 0 (no missing values)
```

In [ ]:
```
# This just checks that the number of rows are the same
if not len(X) == len(y):
    raise ValueError("The number of rows in X and y are different")
else:
    print("X and y have the same number of rows, which is what we want")
```

Model selection

Sometimes you know which kind of model will best suit your application, but other times you need to try a few different ones. In the full paper we tested three types of model:

1. **Linear regression**: our data are non-linear, so we don't expect this to work well, but it is a good 'benchmark'
2. **Random forest regression**: these handle non-linearity well
3. **XGBoost:** a gradient boosting method that can out-perform random forest

Here, to simplify the code slightly, we will just test **Linear regression** and **Random forest regression**.

Start by creating our scikit-learn `Pipeline` objects. Note: as part of the pipeline we can also use the `StandardScaler` to scale our data using Z-Scores.

In [ ]:
```
# Pipeline for linear regression
lr_model_pipeline = Pipeline(steps=[
    ('scaler',StandardScaler()),             # Step 1: normalise
('scale') the data
    ('linear_regressor',LinearRegression())  # Step 2: run the linear
```

```
regression
])

# Pipeline for random forest
rf_model_pipeline = Pipeline(steps=[
    ('scaler',StandardScaler()),
    ('rf_regressor', RandomForestRegressor(n_jobs = os.cpu_count()))  #
Note: n_jobs lets it run on more cores
])
```

We could do full cross-validation with model selection, but for this example we wont. Instead, we will just create our own train and validate (test) datasets, using the handy `train_test_split` function that scikit-learn provides. We need four datasets: train/test for our *X* data, and train/test for our *y* data:

- `X_train` : a subset of the predictor data (weather, built environment, etc.) that we use to train the model
- `y_train` : a subset of the real footfall counts data that we use when training the model (the model will use the `X_train` data to try to make predictions that match these `y_train` data).
- `X_test` : a subset of the predictor data we use to *test* the model afterwards
- `y_test` : a subset of the real data that we use to test how well the model worked after we have finished training it.

Run the code below to have scikit-learn create test and train datasets.

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

Now we try fitting the linear regression and random forest models to those data (fit it to the training dataset and then test its accuracy on the test data)

Run the code below to fit the linear regression to the training data.

In [ ]:
```
lr_model_pipeline.fit(X_train, y_train)
```

Now do the same for the random forest pipeline.

Write the code below to fit the random forest regression to the training data.

In [ ]:
```
# ANSWER
rf_model_pipeline.fit(X_train, y_train)
```

We have trained our models. Let's see which one worked best by seeing how well it can predict footfall, using the data in `X_test` , which it hasn't seen before, to predict footfall in `y_test` .

We will start with the linear regression:

In [ ]:
```
# Create a new dataset of model predictions, giving the model the X_test
data
lr_predict = lr_model_pipeline.predict(X_test)

# Now calculate two error metrics to see how well it did
```

```
lr_rmse = np.sqrt(mean_squared_error(y_test, lr_predict))
lr_r2 = r2_score(y_test, lr_predict)
print("The linear regression model got rmse and r2 scores of:", lr_rmse,
lr_r2)
```

OK, so we have used the fitted linear regression model to make predictions and calculated how good those predictions were when compared to some hourly counts.

Fill in the code below to make predictions for the random forest model (call them `rf_predict`, as we use that name in the graph below) and calculate how well the model worked.

In [ ]:
```
# ANSWER
# Create a new dataset of model predictions, giving the model the X_test
data
rf_predict = rf_model_pipeline.predict(X_test)

# Now calculate two error metrics to see how well it did
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_predict))
rf_r2 = r2_score(y_test, rf_predict)
print("The random forest model got rmse and r2 scores of:", rf_rmse,
rf_r2)
```

Which model was better??

Fill in the markdown cell below explaining which model performed better and how you know this.

ANSWER: The random forest is much better. It has a much lower RMSE (about 220 people per hour, compared with about 640 for linear regression) and a much higher R^2 (about 0.94 compared with about 0.47). The exact values will differ slightly each time the notebook is run, because the train/test split is random. So its predictions are typically much closer to the real counts, and it explains far more of the variation in footfall.

We can also have a look at how the footfall predictions compared to the real footfall counts, to see if there are any systematic under- or over-predictions. If the predictions are perfect, then all points should sit on the $y=x$ line. This isn't strictly necessary at this stage but we will do more of this kind of analysis after optimising the final model.

Read the comments and run the code below, we will return to this later. Does this support your assessment of which model is better?

ANSWER: Yes. The random forest predictions lie close to the $y=x$ line, whereas the linear regression predictions are widely spread and some of them are negative, which is impossible for footfall.

In [ ]:
```
# The graphing is easier if we have all of our data (test data and the
predictions) in a single temporary dataframe.
# We can build the dataframe by passing it a dictionary object that we
```

```
build on the fly:
temp_df = pd.DataFrame({
    'actual': y_test, # The test data ('real' values)
    'linear_regression': lr_predict, # Linear regression predictions
    'random_forest': rf_predict  # Random forest predictions
})
temp_df = temp_df.sample(10000)  # Use a random sample of 10,000 rows,
otherwise the KDE plots are very slow

lr_graph = sns.jointplot(data=temp_df, x='actual',
y='linear_regression', kind='kde', fill=True, height=4)  # Linear
regression
lr_graph.ax_joint.axline((0, 0), slope=1, linestyle='--', linewidth=1)
# Add a line at y = x

rf_graph = sns.jointplot(data=temp_df, x='actual', y='random_forest',
kind='kde', fill=True, height=4)  # Random forest
rf_graph.ax_joint.axline((0, 0), slope=1, linestyle='--', linewidth=1)
# Add a line at y = x
```

Hyperparameter Tuning

The random forest model performed much better than the linear regression model, so we will use that for the rest of the analysis.

Now that we have selected our model, we can tune the hyperparameters to see if we can make it fit even better. There are a lot of hyperparameters that will influence how well a model works on a particular dataset. For example, the RandomForestRegressor documentation lists a large number of parameters that might influence how well it works. Here are just two:

- `n_estimators` : the number of trees in the forest (default 100). More trees can increase accuracy but can make it take longer fit and use more computer memory.
- `max_depth` : the maximum depth of each tree (default None). Smaller trees are simpler and less susceptible to over-fitting. The default, *None* allows them to grow fully and risks overfitting.

To add these to our pipeline we need to prepare a dictionary in the following format:

```
{'model_name__parameter_name': [ list_of_values ] }
```

We called our model 'rf_regressor' when we created the pipeline, so we can specify parameters as follows:

```
parameters = {
    'rf_regressor__n_estimators': [50, 100],
    'rf_regressor__max_depth': [None, 10],
}
```

Edit the code below to add one additional parameter of your choice to the test

In [ ]:
```
parameters = {
    'rf_regressor__n_estimators': [50, 100],
```

```
    'rf_regressor__max_depth': [None, 10],
    'rf_regressor__min_samples_leaf': [1, 5],  # ANSWER: any other
parameter from the documentation will do
}
```

There are a few different methods that the algorithm can use to try out the different parameters. Here we use Grid Search Cross Validation. This basically tries all possible parameter combinations. If we have loads of parameters we might try a cleverer search algorithm such as the Bayes Search CV that tries guess which parameter combinations will work well and not bother testing those that wont.

Read the code below and run it to prepare the Grid Search Cross Validation.

In [ ]:
```
grid = GridSearchCV(
    estimator=rf_model_pipeline,        # The pipeline we created
earlier
    param_grid=parameters,              # Our parameters
    scoring='r2',                       # How to calculate error
('neg_root_mean_squared_error' may be better).
    cv=KFold(n_splits=2, shuffle=True), # We want to do K-fold cross
validation and randomise the order of the data
    n_jobs=os.cpu_count(),              # If possible, run the job over
lots of the computer's cores
    verbose=2                           # Ask it to print some messages
while it's running (increase to 3 for more detail)
)
```

Now we are ready to go. We call

`grid.fit(X_train, y_train)`
to run the grid search. Just for interest we also record the start and end time so we can see how long it takes to run.

Run the code below and go and make a cup of tea while you're waiting. When I ran this on colab it took about 10 mins, but this may vary depending on the number of parameter combinations you have.

In [ ]:
```
start = time.time()
grid.fit(X_train, y_train)
end = time.time()

print("Grid search finished in", (end - start)/60, "minutes.")
```

Phew. It's done. Let's see which parameters were best:

In [ ]:
```
print("Best params:", grid.best_params_)
```

Model Evaluation

We have now selected our model and tuned its hyperparameters. The final thing is to see how well it can predict footfall on data it has not seen before.

When the grid search finished, `GridSearchCV` automatically fitted a model to our training data using the best hyperparameters it found. We can get that model with `grid.best_estimator_`. This is our 'best' model:

In [ ]:
```
best_model = grid.best_estimator_  # The best model, already fitted to the training data
```

Now we can use that model to make predictions on our *test* data. It has not seen these data before. We evaluate it in the same way that we evaluated the models during model selection:

In [ ]:
```
# Use the model to make predictions from our test data
best_model_predictions = best_model.predict(X_test)

# Calculate the error by comparing our test data to the model predictions
rmse = np.sqrt(mean_squared_error(y_test, best_model_predictions))
r2   = r2_score(y_test, best_model_predictions)

print("Test RMSE:", rmse)
print("Test R^2:",  r2)
```

We can also graph the predictions against the real data as we did before

In [ ]:
```
temp_df = pd.DataFrame({
    'actual footfall': y_test, # The test data ('real' values)
    'best model predictions': best_model_predictions  # Random forest predictions
})
temp_df = temp_df.sample(10000)  # A random sample, to keep the KDE plot quick

best_graph = sns.jointplot(data=temp_df, x='actual footfall', y='best model predictions', kind='kde', fill=True, height=4)  # Random forest
best_graph.ax_joint.axline((0, 0), slope=1, linestyle='--', linewidth=1)
# Add a line at y = x
```

Interpreting the model

As well as predicting footfall, we would like to know *what drives it*. A random forest records how much each predictor helped its predictions. This is called **feature importance**. The code below draws a bar chart of the importance of each predictor: a longer bar means the model relied on that variable more.

Remember that importance is not cause and effect. For example, the built-environment variables (trees, landmarks, etc.) are the same for every hour at a sensor, so they partly act as a label for each location. And time of day is split across two columns (`Sin_time` and `Cos_time`), so you need to add them together.

Run the code below. Which variables matter most? Is that what you expected?

ANSWER: Time of day matters most (add `Sin_time` and `Cos_time` together), followed by some of the built-environment counts such as trees and landmarks. Rainfall and the holidays matter much less. The high scores for trees and landmarks do not mean that these features attract people: they are fixed for each sensor, so the model mainly uses them to tell the sensors apart.

In [ ]:
```
# Get the importance of each predictor from the random forest part of our pipeline
importances = pd.Series(best_model['rf_regressor'].feature_importances_, index=predictor_columns)

# Sort them and draw a horizontal bar chart
importances.sort_values().plot.barh(figsize=(6, 6))
plt.xlabel("Importance")
plt.title("Which variables does the random forest rely on?")
plt.tight_layout()
```

Using the model to predict footfall

Now that we have a working model, there are loads of things we can do with it. For example:

- Analyse the errors to see where footfall varied from what we would have expected and try to understand why
- Plug in new *X* values to predict what footfall might be like next week
- Quantify how successful previous events were (did we see an increase or decrease in footfall given the weather conditions on the day?)

To finish, let's look at footfall on 1st January 2015. We will make a prediction of footfall at every hour on that day for one sensor in particular (sensor 5) and see how it compared to real footfall.

The code below should just run. Read it and please ask if you don't understand any of it. It looks complicated but there is nothing you haven't come across before.

In [ ]:
```
# Create X and y variables for 1st Jan 2015 (we use new names so we don't overwrite our X and y)
X_jan1 = location_counts.loc[
    (location_counts['datetime'].dt.date == pd.to_datetime('2015-01-01').date()) & (location_counts['sensor_id']==5),
    predictor_columns]

y_jan1 = location_counts.loc[
    (location_counts['datetime'].dt.date == pd.to_datetime('2015-01-01').date()) & (location_counts['sensor_id']==5),
    'hourly_counts']
```

In [ ]:
```
# Predict footfall on that date
prediction = best_model.predict(X_jan1)
```

In [ ]:
```
plt.figure(figsize=(8,4))
```

```
sns.lineplot(x=range(len(y_jan1)), y=y_jan1, label='Actual')
sns.lineplot(x=range(len(y_jan1)), y=prediction, label='Prediction')

plt.xlabel("Time of day (hours)")
plt.ylabel("Hourly counts")
plt.title("Actual vs Prediction")
plt.tight_layout()
```

You might be surprised that the prediction follows the very high footfall just after midnight (New Year's Eve celebrations) quite closely. There is nothing in our predictors that tells the model about New Year's Eve. The reason is **data leakage**: we split the data randomly, so most of these 24 hours were in the training data and the model has already seen the real counts. If you trained the model without the data for 1st January, its predictions for the early hours would be much lower than the real footfall.

Extra: A fairer test -- splitting the data by time

Recall from the lecture that splitting the data randomly can cause **data leakage**. Our footfall data are chronological, so with a random split the model is trained on hours that come *after* some of the hours it is tested on. In effect, it has seen the future. A fairer test is to train the model on the earliest 80% of the data and test it on the most recent 20%.

The code below sorts the data by time and then uses `train_test_split()` with `shuffle=False`, so the rows stay in time order. It then makes a new copy of our best model (`clone()` makes an unfitted copy with the same hyperparameters), fits it to the earlier data only, and calculates the error on the later data. We use new variable names throughout so that the data and model from earlier in the notebook are not changed.

Run the code below. How do the RMSE and R^2 compare with the ones from the random split above? Which do you think gives a more realistic idea of how well the model would predict footfall next week?

ANSWER: The chronological split gives a higher RMSE and a lower R^2 (in our run, an RMSE of about 310 and an R^2 of about 0.88, compared with about 220 and 0.94 for the random split). It is the more realistic test, because when we use the model to predict next week it will not have seen any data from that week. The random split gives an optimistic estimate because the model was trained on hours either side of the ones it was tested on.

In [ ]:
```
from sklearn.base import clone

# Random forests use a lot of memory. We no longer need the grid search
or the random forest
# from the model selection step, so delete them to make room for fitting
another model.
# Jupyter also keeps a copy of earlier cell outputs (which can include
fitted models), so clear those too.
```

```
%reset -f out
del grid, rf_model_pipeline

# Sort the data by time (into a new dataframe) and create X and y from
it, in time order
location_counts_sorted = location_counts.sort_values('datetime')
X_sorted = location_counts_sorted.loc[
location_counts_sorted['datetime'] < '2020', predictor_columns]
y_sorted = location_counts_sorted.loc[
location_counts_sorted['datetime'] < '2020', 'hourly_counts']

# shuffle=False keeps the rows in order, so the test data are the most
recent 20% of the hours
X_train_time, X_test_time, y_train_time, y_test_time =
train_test_split(X_sorted, y_sorted, test_size=0.2, shuffle=False)

# Make a new copy of the best model, fit it to the earlier data only,
and test it on the later data
time_model = clone(best_model)
time_model.fit(X_train_time, y_train_time)
time_predictions = time_model.predict(X_test_time)

print("Chronological split RMSE:",
np.sqrt(mean_squared_error(y_test_time, time_predictions)))
print("Chronological split R^2:", r2_score(y_test_time,
time_predictions))
```

That's it! For those who are really keen:

Stretch activity: create your own hypothetical *X* data for a single sensor and hour, and make a prediction on what the footfall might be.

In [ ]:

```
# ANSWER (stretch activity)
# The easiest way to get all 23 predictors in the right format is to
copy one existing row
new_X = X_test.iloc[[0]].copy()

# Then change the values we are interested in, e.g. a very wet day
new_X['rainfall_mm'] = 20

print("Predicted footfall with no change:",
best_model.predict(X_test.iloc[[0]]))
print("Predicted footfall with 20mm of rain:",
best_model.predict(new_X))
```